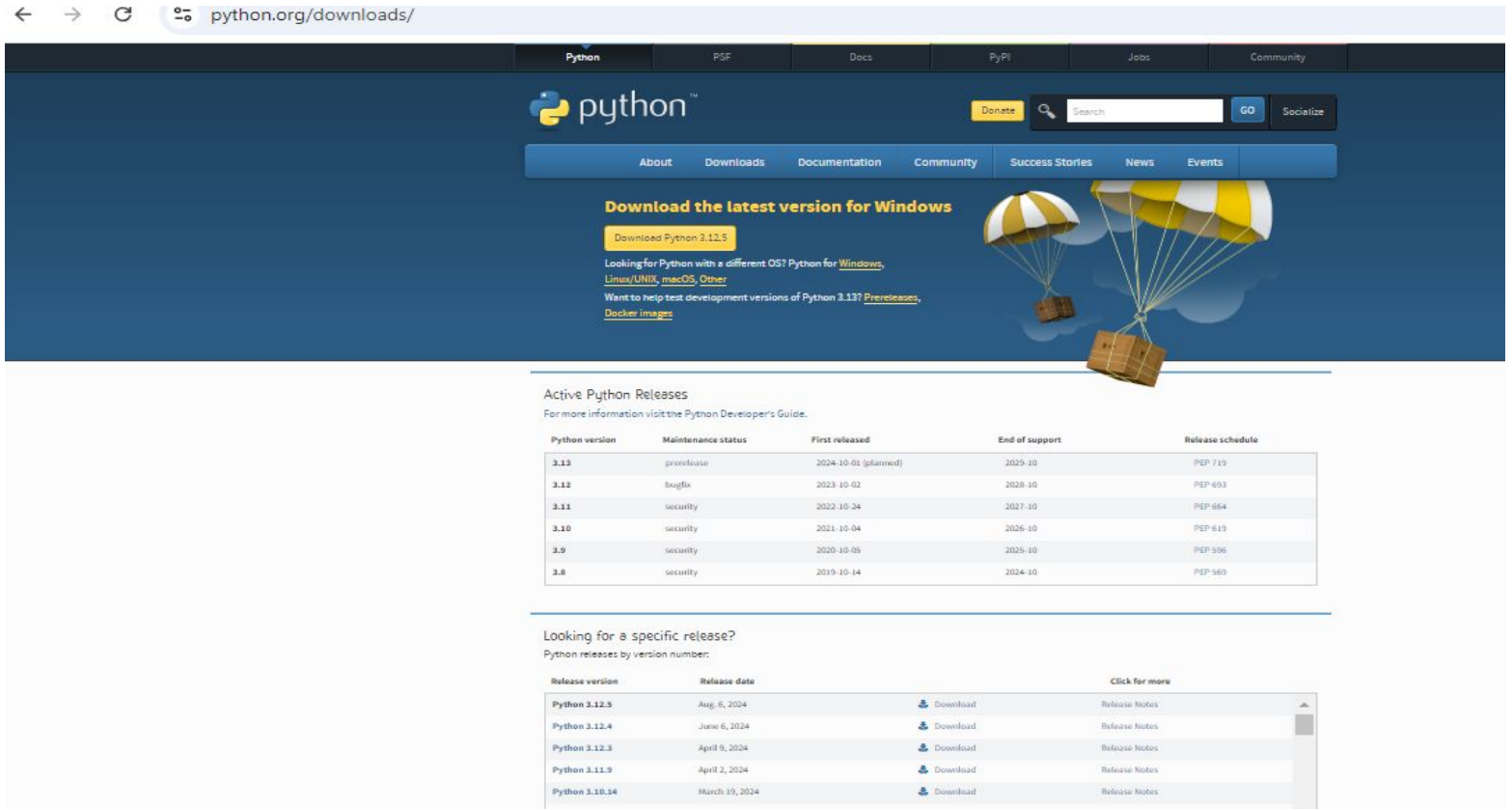
Installation of Python & packages

By PrudviKrishna

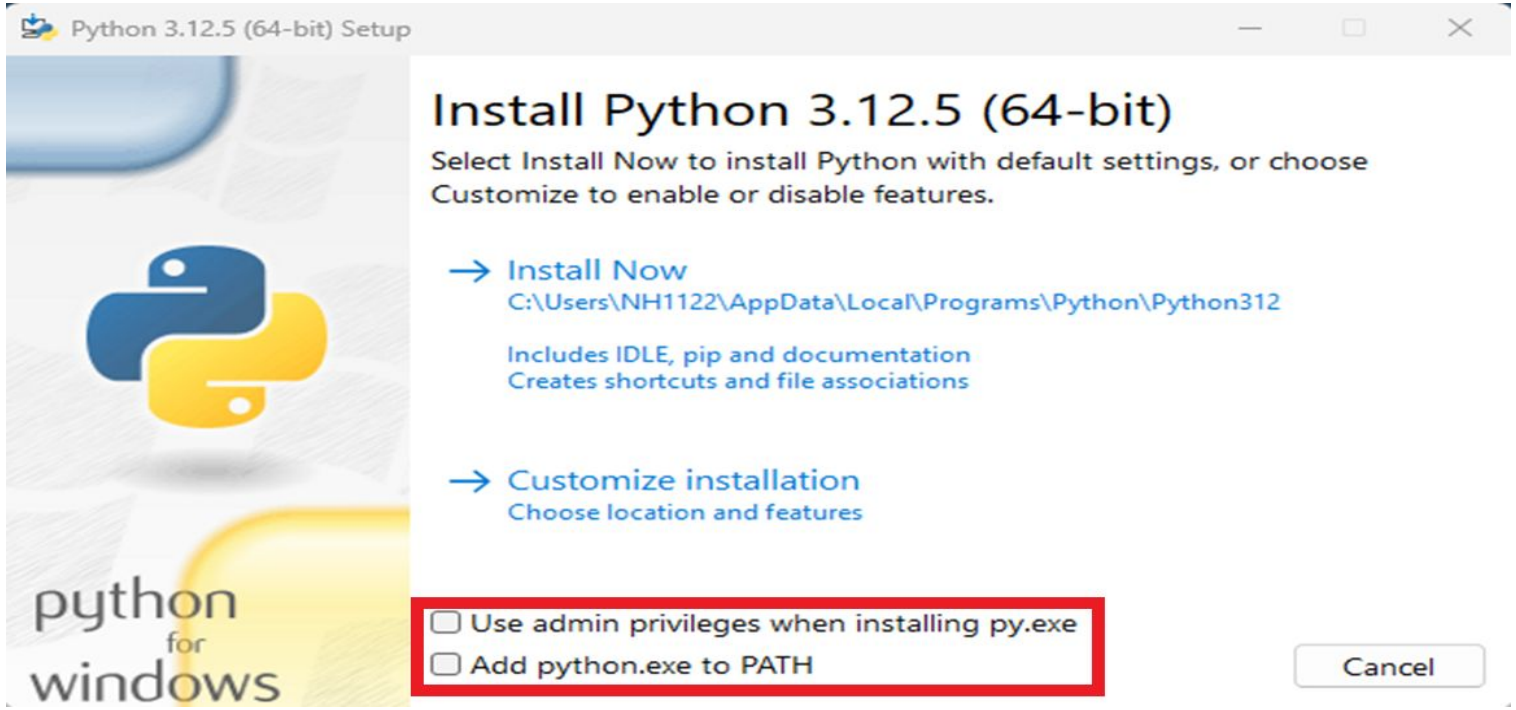
Installation of Python on Windows

- Download Python from:
  https://www.python.org/downloads/release/python-3125/



Installation of Python on Windows

- Click on install now and don't forget to check the “Add python.exe to PATH”
- Once installation is done. Go to run and open “cmd” and type py in the terminal to validate the installation.
- To know the python version installed py --version or python –version or py –V
- To know the path of python where it is installed **“where python”**



Python Distributions

Python distributions, bundle Python itself along with a collection of libraries and tools to simplify the setup of a Python environment. Following are some of the python distributions.

1. **Anaconda**:
   - A popular distribution focused on data science and machine learning.
   - Comes with many pre-installed libraries like NumPy, pandas, SciPy, and Jupyter Notebook.
   - **Ideal For**: Data science, machine learning, scientific computing.
2. **Miniconda**:
   - A lightweight version of Anaconda with only the essentials (Python and Conda).
   - Ideal For: Those who want more control over package installations.
3. **ActivePython**:
   - A Python distribution provided by ActiveState, aimed at enterprise environments.
   - Includes pre-built Python libraries for machine learning, data science, and DevOps.
   - **Ideal For**: Enterprise use cases where stability and security are critical.
4. **WinPython**:
   - A portable distribution of Python for Windows.
   - Comes pre-packaged with scientific libraries like SciPy, NumPy, and matplotlib.
   - **Ideal For**: Users who need a Python environment on Windows without admin rights.
5. **Enthought Canopy** (Discontinued):
   - Previously used for scientific and analytic computing.
   - Was targeted at academia and research.

Installation of Anaconda

- Anaconda is an **open-source distribution** of the Python programming language that's used for data science, machine learning, and artificial intelligence applications
- https://www.anaconda.com/download
- Anaconda comes with a curated collection of pre-installed packages, including SciPy, Matplotlib, Pandas, and NumPy and many common Python packages for data science
- Anaconda installation documentation
  - Windows : https://docs.anaconda.com/anaconda/install/windows/
  - MacOS : https://docs.anaconda.com/anaconda/install/mac-os/
  - Linux : https://docs.anaconda.com/anaconda/install/linux/

Package Managers

Package Mangers are used to install, update, and manage Python packages and their dependencies.

1. **PIP (PIP Installs Packages)**:
   - The most widely used Python package manager, part of Python's standard library since Python 3.4.
   - **Ideal For**: General Python development and package management.

2. **Conda**:
   - Comes with Anaconda and Miniconda distributions.
   - Manages both Python and non-Python dependencies (e.g., R, C++ libraries).
   - **Ideal For**: Data science, machine learning, and environments with complex dependencies.

3. **Poetry**:
   - A newer package manager focused on simplifying dependency management and packaging.
   - Automatically resolves dependencies and generates pyproject.toml files.
   - **Ideal For**: Modern Python projects that need structured dependency management and easy packaging.

Package Managers

Action	pip Command	conda Command
Install a package	pip install package_name	conda install package_name
Install a specific version	pip install package_name==version_number	conda install package_name=version_number
Upgrade a package	pip install --upgrade package_name	conda update package_name
Uninstall a package	pip uninstall package_name	conda remove package_name
install from a specific channel	N/A	conda install -c channel_name package_name
Package installed path	Pip show packagename	
List of all packages installed	pip list	conda list
Move all packages to a file	pip freeze > requirements.txt	conda env export > environment.yml
Install multiple packages	pip install -r requirements.txt	conda install package1 package2
Install from a file	pip install -r requirements.txt	conda env create -f environment.yml

PyPI and Channels

PyPI (Python Package Index)

PyPI is the official repository for Python packages, hosting thousands of third-party libraries and tools.

Package Repository: Developers publish their Python packages to PyPI, making them available for anyone to install using pip.

Search and Explore: Users can search for packages by name or functionality through the PyPI website (https://pypi.org)

Conda Package Sources(Channels) :

- Anaconda Repository (default)
- Conda-Forge(conda packages maintained by the conda-forge community)
- Bioconda (large collection of bioinformatics tools, libraries, and utilities

PEP (Python Enhancement Proposals)

PEP (Python Enhancement Proposals) are design documents that provide information, proposals, and guidelines on new features, changes, or best practices for the Python language. https://peps.python.org/

Some of PEP guidelines like

- **PEP 8** (code style guide)
- **PEP 440** (versioning)
- **PEP 517** (build systems) etc.

Note: PEP standards are not mandatory for adding a module to PyPI (Python Package Index). For core Python (like CPython), it's mandatory.

Installation of libraries on Windows

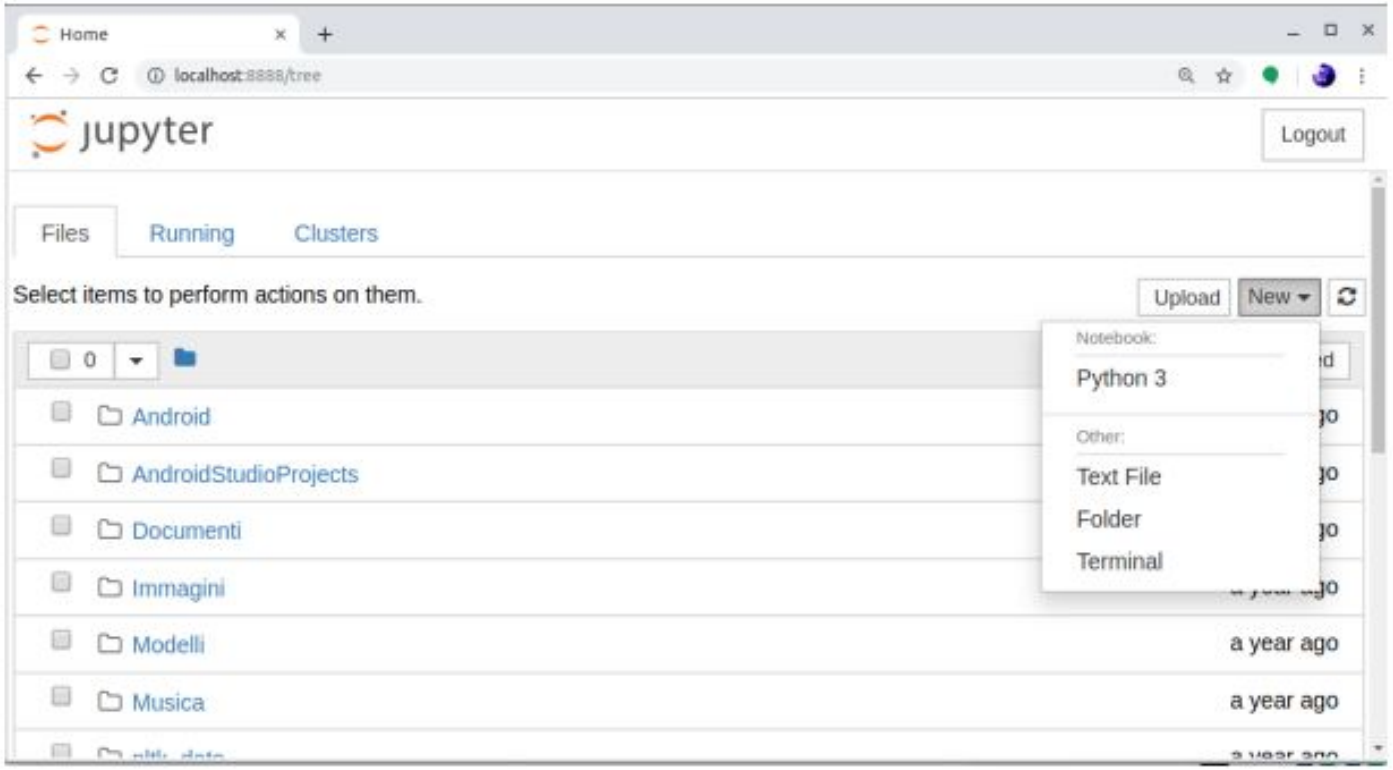
- Open the command prompt and type:
  - pip install ipython
  - pip install jupyter

These commands will install ipython and jupyter notebook

- Installing libraries
  - Install the following libraries with the pip3 command (on your command prompt)
    - pip install numpy
    - pip install pandas
    - pip install matplotlib
    - pip3 install scikit-learn

Validate installations on Windows

- Test your installation by typing in your anaconda command prompt
  jupyter notebook
- You should obtain in your browser a new Jupyter session



Installation of Python on Ubuntu

- The main programs for running python can be installed via **apt-get** on your terminal
  - sudo apt-get install python3
  - sudo apt-get install python3-pip
  - pip3 install ipython
  - pip3 install jupyter

- Installing libraries
  - Python language is provided with many useful libraries
  - Install the following libraries with the pip3 command:
    - pip3 install numpy
    - pip3 install pandas
    - pip3 install matplotlib
    - pip3 install scikit-learn

Installation of Python on MacOS

- Open your Terminal app
- Install Command Line Tools (CLT) for Xcode
  - $ xcode-select --install (not needed if you already have Xcode)
- Install Homebrew from https://brew.sh
  - It's the standard de-facto macOS package manager
  - The command below downloads and installs it

$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

- Now, the main programs can be installed via **Homebrew** on your terminal:
  - $ brew install python (installs python3 and pip3)
  - $ pip3 install jupyter (installs jupyter and ipython)

Note: you might need to restart the terminal to see the changes

Installation of packages on MacOS

- Installing libraries
  - Install the following libraries with the **pip3** command:
    - $ pip3 install numpy
    - $ pip3 install pandas
    - $ pip3 install matplotlib
    - $ pip3 install scikit-learn
- Additional hints
  - Many executables can be installed via Homebrew

    $ brew install wget
  - You can search or have information about a program

    $ brew search <program-name>

    $ brew info <program-name>
  - Or list the installed ones

    $ brew list
  - The Anaconda distribution can be installed via Homebrew

    $ brew cask install anaconda

Integrated Development Environments(IDEs)

IDE stands for Integrated Development Environment, which is a software application that helps programmers write, test, build, and package software code.

- On machine
  - ✔ Visual studio code (VS Code)
  - ✔ Jupyter Notebook
  - ✔ PyCharm
  - ✔ Spyder
  - ✔ Notepad++
  - ✔ IDLE (default with Python installation)
- Cloud/Web based
  - ✔ Google Colab
  - ✔ Kaggle etc.
  - ✔ GitHub Codespaces

Python Vs IPython

Python Interpreter (Standard Python REPL) Read-Eval-Print Loop (REPL) environment.

IPython is an interactive shell that is built with python. It makes it more interactive by adding features like syntax highlighting, code completion, magic commands etc.

By default, Jupyter Notebooks use the IPython kernel to execute Python code.

Jupyter can also use a plain Python interpreter (or any other interpreter) as a kernel, but will lose IPython's enhanced features

Magic command %, %% examples:

In [1]: x = 10

In [2]: y = 20

In [3]: %save my_script.py 1-2

This saves lines 1 to 2 of the session to a file named my_script.py

To Execute the python script within ipython shell

%run <filename.py>

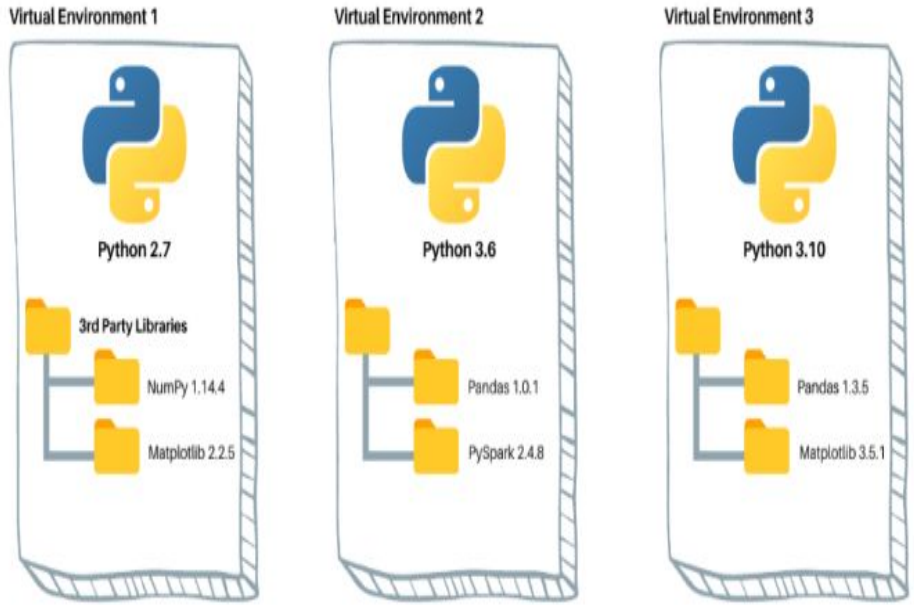
Python Environments

Using two machines to avoid conflicts from different data science setups isn't feasible or cost-effective; instead, virtual environments is the solution.

A Python virtual environment consists of two essential components:

- The Python interpreter that the virtual environment runs on
- A folder containing third-party libraries installed in the virtual environment.

❖ Ways to install and manage virtual environments
- venv (Built-in Python)
- virtualenv (Third-Party Tool)
- conda (Anaconda/Miniconda) Module)



Python Environments

Tool	Description	Installation	Create Env	Activate	Deactivate
venv	Pre-installed with Python venv is a module included with Python 3.3 and later	No separate installation is required	python -m venv myenv	Windows: myenv\Scripts\activate macOS/Linux: source myenv/bin/activate	deactivate
virtualenv	virtualenv is a third-party tool. It works with both Python 2 and 3.	pip install virtualenv	virtualenv myenv	Windows: myenv\Scripts\activate macOS/Linux: source myenv/bin/activate	deactivate
conda	conda is a package manager and environment management system from the Anaconda distribution	Requires Anaconda or Miniconda to be installed.	Created at ~/anaconda3/envs/ conda create --name myenv In current folder conda create -p myenv python==3.12	conda activate myenv conda activate myenv/	conda deactivate

Example commands

1. First make a project folder, and create a virtual environment inside it	8. list of packages installed on an environment in a correct format to reproduce the environment with the exact package versions
$ mkdir myfolder	$ pip freeze
$ python3 -m venv myfolder/myvenv -- for a default version $ py -3.11 –m venv myfolder/myvenv -- for specific version	9. Exporting the package list into the requirements.txt file
2. Activating a Python Virtual Environment	$ pip freeze > requirements.txt
$ source myfolder/myvenv/bin/activate	10. To reproduce the virtual environment.
3.Location of the Python interpreter within the virtual environment	$ pip install -r requirements.txt
(myvenv) $ which python	11. Deactivating a Python Virtual Environment
/Users/username/myfolder/myvenv/bin/python	$ deactivate
4. Register and display the environment as kernel in jupyter notebook	12. Deleting a Python Virtual Environment
python -m ipykernel install --user --name=myenv --display-name "Python (myenv)"	rm -rf /path/to/myenv # Linux/Mac rmdir /S /Q \path\to\myenv # Windows conda remove --name env_name --all
5. Check the pre-installed packages on the virtual environment	13. using conda create environments with specific python versions
(myvenv) $ pip list	$ conda create -n name_of_the_folder python=3.12
6. Upgrade pip to the latest version	14. Like requirement.txt file, in conda environment, we use environment.yml files. install all the packages with a specific version, we need to use the below command.
(myvenv) $ myfolder/myvenv/bin/python3 -m pip install --upgrade pip	$ conda env create -f environment.yml
7. Installing any package in the environment	15. To list the available environments in conda
pip install 'pandas>0.25.3'	$ conda env list

Thank you